Task 9: Spring MVC Employee Management

Friday, March 20, 2026 10:11 PM

Objective

Create a simple web application using **Spring Boot + Spring MVC + Thymeleaf** that:

- Takes Employee ID from user
- Displays employee details in browser

🧰 Step 1: Install Required Software

Install these first:

1. **Java JDK (Version 17 recommended)**
2. **Eclipse IDE (Enterprise Edition)**
3. Internet connection (for dependencies)

🚀 Step 2: Create Spring Boot Project

In Eclipse:

1. Click **File → New → Spring Starter Project**
2. Fill details:
   - Name: springmvc
   - Type: Maven
   - Packaging: Jar
   - Java Version: 17
3. Click **Next**

Add Dependencies:

✓ Spring Web
✓ Thymeleaf
Click **Finish**

📂 Step 3: Check Project Structure

You should see:

```
src/main/java/com/example/demo
   ├── Employee.java
   ├── EmployeeController.java
   ├── EmployeeRepository.java
   └── SpringmvcApplication.java

src/main/resources
   ├── templates/
   │   ├── home.html
   │   └── employee.html
   └── application.properties
```

🤖 Step 4: Create Java Files

✅ 1. Employee.java

```
package com.example.demo;

public class Employee {
   private int id;
   private String name;
   private String dept;

   public Employee(int id, String name, String dept) {
      this.id = id;
      this.name = name;
      this.dept = dept;
   }

   public int getId() { return id; }
   public String getName() { return name; }
   public String getDept() { return dept; }
}
```

✅ 2. EmployeeRepository.java

```
package com.example.demo;

import java.util.Map;
import org.springframework.stereotype.Component;

@Component
public class EmployeeRepository {

   private final Map<Integer, Employee> store = Map.of(
      101, new Employee(101, "Anand", "cse"),
      102, new Employee(102, "Ravi", "it")
   );

   public Employee findById(int id) {
      return store.get(id);
   }
}
```

✅ 3. EmployeeController.java

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
```

```
public class EmployeeController {

    @Autowired
    private EmployeeRepository repo;

    @GetMapping("/")
    public String home() {
        return "home";
    }

    @GetMapping("/employee")
    public String getEmployee(@RequestParam("id") int id, Model model) {
        Employee emp = repo.findById(id);

        if (emp == null) {
            model.addAttribute("error", "Employee not found: " + id);
        } else {
            model.addAttribute("emp", emp);
        }

        return "employee";
    }
}
```

🌐 Step 5: Create HTML Files (Thymeleaf)

✅ 1. home.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Home</title>
</head>
<body>

<h2>Employee Management</h2>

<form action="/employee" method="get">
    <input type="number" name="id" required>
    <button type="submit">Search</button>
</form>

<p>Try: 101, 102</p>

<p style="color:red" th:text="${error}"></p>

</body>
</html>
```

What is this?

✅ 2. employee.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
   <title>Employee Details</title>
</head>
<body>

<h2>Employee Details</h2>

<div th:if="${emp != null}">
   <p><b>ID:</b> <span th:text="${emp.id}"></span></p>
   <p><b>Name:</b> <span th:text="${emp.name}"></span></p>
   <p><b>Department:</b> <span th:text="${emp.dept}"></span></p>
</div>

<div th:if="${error != null}">
   <p style="color:red" th:text="${error}"></p>
</div>

<br>
<a href="/">Search Another Employee</a>

</body>
</html>
```

What is this?

▶ Step 6: Run the Project

1. Open:

   SpringmvcApplication.java
2. Right-click → **Run As → Java Application**
3. Wait for console message:

   Tomcat started on port 8080

🌍 Step 7: Open in Browser

Go to:

http://localhost:8080

🧪 Step 8: Test

Enter:

101

👉 Output:

ID: 101
Name: Anand
Department: cse

❌ Error Case

Enter:

999

👉 Output:

Employee not found